AOE2 CIV EXPLORER

Code documentation

What each part of the code does, and why

19 October 2026 · 14 chapters · 159 functions

This document explains the code of AoE2 Civ Explorer. Each file is a chapter. In the script chapters, each function defined at the top level of the script is a subchapter, saying what it does and why in at most 240 characters. Helpers defined inside a function are part of that function's text.

"Exported" marks a function other scripts use; "internal" marks one only its own script uses. The two JSON chapters at the end are kept short.

Contents

1	`index.html`	9 parts
2	`style.css`	11 parts
3	`script.js`	1 part
4	`js/datahandler.js`	35 functions
5	`js/model.js`	22 functions
6	`js/ui.js`	32 functions
7	`js/tests/run.js`	4 functions
8	`js/tests/helpers.js`	25 functions
9	`js/tests/static.js`	6 functions
10	`js/tests/behaviour.js`	13 functions
11	`js/tests/interface.js`	14 functions
12	`js/tests/audit.js`	8 functions
13	`data.json`	
14	`locales/de.json`	

1 `index.html`

The page's structure, and all of its HTML (Rule 2). It holds no text of its own: the scripts fill in every word in the chosen language.

1.1 Head

Sets the language, character set, viewport, light and dark colour schemes, title and description, and loads `style.css`, the icon and `script.js` as a module. No script or style is written in the page itself.

1.2 Header

Holds the title and the cogwheel Options button (Req. 2, 6). The cogwheel is a text character, not an image (Rule 9), with a hidden label for screen readers. Hidden until the page has started.

1.3 Messages

A noscript line for browsers without JavaScript, and a load error shown if `data.json` cannot be loaded, so a visitor is never left with a blank page.

1.4 Tree pane

The left pane (Req. 3): heading, Clear, Expand all and Collapse all, the Find field with its live match count, the must have and must lack legend, and the empty tree list the code fills in.

1.5 Civilisations pane

The right pane (Req. 4): heading, live count, the Filters list (Req. 22), the civilisation list and the hint shown when it is empty. It scrolls on its own, so the page never does (Rule 12).

1.6 Footer

Names and links the data source and its date (Req. 5). The code writes the sentence around the link in the chosen language.

1.7 Options window

A dialog (Req. 7, 8) with its title, the language label, button and list, the Include Unique and Universal box with its explanation, and Close. The language list takes the settings' place while open.

1.8 Report window

A dialog for the reports of the tests and the updater: title, live status, result list, download links and Close. Always in English, so it is marked lang="en".

1.9 Templates

Markup for tree rows, Filters buttons, civilisations, language options, report lines, downloads and test frames. The code copies these instead of writing HTML, so all HTML stays in `index.html` (Rule 2).

2 `style.css`

How the page looks, and all of its CSS (Rule 2): colours, sizes and layouts for big and small screens, in light and dark mode.

2.1 Colours

Named colours for light mode, redefined for dark mode with prefers-color-scheme. Text and borders meet WCAG AAA contrast, which the tests measure in both modes.

2.2 Base

Box sizing, a full-height page that never scrolls (Rule 12), the system font with 1.5 line spacing (WCAG 1.4.8), a 3 px focus outline (WCAG 2.4.13), and a class that hides text from sight but not from screen readers.

2.3 Header and Options

Lays out the title and cogwheel, and the Options window as a grid of fixed height, so opening the language list never resizes it (Rule 18). The current language is marked with a bar and bold text.

2.4 Panes

Two panes side by side from 720 px wide, stacked below that. Each pane's head stays put while its list scrolls inside it.

2.5 Controls

Buttons and the Find field are at least 44 by 44 px (WCAG 2.5.5), with visible borders and a hover colour.

2.6 Tree

Indented rows with dotted guide lines, 44 px rows, turning arrows, highlighted matches and pressed rows, and bold top-level and marked names. Counts never wrap.

2.7 Check boxes

Drawn with borders, not images (Rule 9): green with a tick for must have, red with a cross for must lack, a dash for mixed. In forced colours mode they use the system's colours.

2.8 Filters and list

Filters buttons as rounded chips, and the civilisation list in columns that never split a name.

2.9 Dialogs

Size, padding, border and backdrop of the Options and report windows.

2.10 Small screens

Below 500 px high, or narrow and short, the content scrolls as one; below 400 px wide, gaps shrink. This keeps the page usable at 400% zoom (WCAG 1.4.10).

2.11 Test helpers

Sizes of the frames the tests open, and the wider text spacing they apply (WCAG 1.4.12). Used only while testing.

3 `script.js`

Where the page starts, in the main folder. It defines no functions; its top-level code runs once.

3.1 Start-up

Loads `data.json`, then the visitor's language (every language for `?test` or `?update`), then calls `start()`. With `?test` it loads the tests, with `?update` it runs the updater. If loading fails, it shows the load error.

4 js/datahandler.js

Everything about data: loading, the chosen language and its text, sets of civilisations, saved choices, and the updater.

4.1 `attempt(action)` EXPORTED

Runs a function and returns null instead of an error. Used where the browser may refuse, such as storage in a private window, so a refusal never breaks the page.

4.2 `recall(key)` EXPORTED

Reads a saved choice from the browser's storage, or null. Remembers the visitor's language and Include choice between visits.

4.3 `remember(key, value)` EXPORTED

Saves a choice in the browser's storage, or removes it when the value is null. Failures are ignored, as the page works without storage.

4.4 `load(url)` EXPORTED

Fetches a file as text and returns a promise. It uses XMLHttpRequest because, unlike fetch, it also reads files when the page is opened from disk, as the tests in CI do.

4.5 `fill(template, values)` EXPORTED

Replaces `{name}` placeholders in a text with values, leaving unknown ones alone. All text lives in the data with placeholders, so the code never builds sentences itself.

4.6 `treeKeys(nodes, key)` EXPORTED

Lists every item (or group) name in a tree, top to bottom. Used to check the tree, to find groups that need text, and by the updater to know which items to look up.

4.7 `useLanguage(code)` EXPORTED

Switches the active language: its text, language tag, sorting, plural rules and number format. Everything that shows, sorts or counts text reads these, so one call changes them all.

4.8 `nameOf(item)` EXPORTED

Returns a tree item's or group's name in the current language, or its key if missing. Used wherever an item is shown or searched.

4.9 `civName(civ)` EXPORTED

Returns a civilisation's name in the current language, or its key if missing. Keeps the civilisation list translated.

4.10 `plural(key, n)` EXPORTED

Picks the right form of a counted text, such as "1 civilisation" or "5 civilisations", by the language's plural rules, and fills in the formatted number.

4.11 `plain(text)` EXPORTED

Lower-cases a text and strips accents, spaces and punctuation, so Find matches names however they are typed, in any language.

4.12 `localeFile(code)` EXPORTED

Returns the path of a language's file, such as locales/`de.json`. Keeps the pattern in one place for loading and for the updater's downloads.

4.13 `unpack(file)` EXPORTED

Turns `data.json` as stored into the shape used in memory, where overrides and text are kept per language, so other languages can be added beside English.

4.14 `addLanguage(set, code, file)` EXPORTED

Adds one language file's overrides and text to the data in memory. Used when a language loads, and when a test reads back the files the updater would write.

4.15 `useData(raw)` EXPORTED

Reads `data.json` and gives each civilisation one bit, so a set of civilisations is a single number that & and | combine quickly.

4.16 `loadLanguage(code)` EXPORTED

Loads a language's file unless it is loaded already. A visitor downloads only the language shown, not all 17.

4.17 `loadLanguages()` EXPORTED

Loads every language's file. The tests and the updater need them all.

4.18 `pack(set)` EXPORTED

Splits the data in memory back into files: `data.json` with English, and one file per other language. The updater writes these, and a test checks that nothing is lost.

4.19 `preferredLanguage()` EXPORTED

Chooses the language to show: from the address (`?lng=`), the saved choice, the browser's languages, or else English. So shared links, return visits and first visits all fit.

4.20 `bit(civ)` INTERNAL

Returns a civilisation's bit, or stops with an error for an unknown name, so a typo in `data.json` cannot silently drop a civilisation.

4.21 `size(set)` EXPORTED

Counts the civilisations in a set by counting its bits. Behind every number on the page.

4.22 `setOf(rule)` EXPORTED

Turns an availability rule from `data.json` ("all", a has list or a lacks list) into a set. Rules stay short in the file while the code works with sets.

4.23 `civsOf(set)` EXPORTED

Turns a set back into a list of civilisations, in `data.json` order. Used to list the civilisations that fit.

4.24 `isScalar(value)` INTERNAL

Tells whether a value is text, a number, true or false, or null. Helps `toJson()` decide what fits on one line.

4.25 `isFlatList(value)` INTERNAL

Tells whether a value is a list of scalars only, which `toJson()` writes on one line.

4.26 `flat(value)` INTERNAL

Writes a value as one line of JSON, with a space after each comma and colon. The building block of `toJson()`.

4.27 `toJson(value, indent)` EXPORTED

Writes JSON the way `data.json` is laid out: short objects and lists on one line, the rest indented. Keeps the updater's files readable and their changes easy to review.

4.28 `isLower(char)` INTERNAL

Tells whether a character is a lower-case letter. `joinLines()` uses it to tell a hyphenated word from a real hyphen.

4.29 `capitalise(text)` INTERNAL

Upper-cases the first letter of a text, as some names in the game start in lower case.

4.30 `joinLines(text, settings)` EXPORTED

Joins a game text broken with `<br>` into one line, by the language's rules for spaces and hyphens, and tidies spaces. The game breaks names to fit its buttons.

4.31 `localName(strings, node, settings, offset)` INTERNAL

Finds an item's name in a language's game texts. If that name looks abbreviated, it takes the bold name from the item's help text instead, which is written out in full.

4.32 `availabilityOf(owners, civs)` EXPORTED

Writes who has an item in its shortest form: "all", a has list or a lacks list. Keeps `data.json` small; a test checks that every rule is stored this way.

4.33 `missingText(set)`

EXPORTED

Lists, per language, the page texts and group names that are missing. A test and the updater's report use it, so no language shows a blank.

4.34 `refresh(current, techtree, trees, strings)`

INTERNAL

The updater's core: works out the civilisations, who has what, and every name from aoe2techtree.net's data, and lists each change for the report. Stops if the tree names an unknown item.

4.35 `fetchUpdate()`

EXPORTED

Downloads aoe2techtree.net's data, trees and texts, runs `refresh()`, and returns the report and the files that changed. So a game update needs only a browser.

5 js/model.js

The tree and its rules: which items are shown, which are marked, and how marks follow from each other. It never touches the screen.

5.1 `setIncluded(on)` EXPORTED

Sets whether unique and universal items are included. A function, because other scripts cannot change an imported variable themselves.

5.2 `collect(nodes, parent)` INTERNAL

Turns the tree from `data.json` into item objects with their set of civilisations, path, parent and children, in tree order. The page and the tests work with these.

5.3 `whole(item)` EXPORTED

Returns the civilisations that have an item and everything under it, remembering the result. It is the second number on a closed item.

5.4 `prune(nodes)` EXPORTED

Returns the default tree: without unique groups, and without items every civilisation or only one has, unless something under them stays. Keeps the default tree short.

5.5 `rebuild()` EXPORTED

Builds the items again from the full or the default tree, keeping each item's mark by its path. Runs when Include Unique and Universal changes.

5.6 `setMark(item, mark)` EXPORTED

Sets a mark on an item and on everything under it. Used by Clear, by marking a building or group, and to clear an item with all below it.

5.7 `settle()` EXPORTED

Works out every item's mark from the bottom up: an item counts as must have when everything under it does, unless it is optional (Req. 27). So marks follow without being stored.

5.8 `state(item)` EXPORTED

Returns the state of an item with everything under it: "has", "lacks", none, or "mixed" when they differ. Decides the dash in a box, and what a click on a building does.

5.9 `isUnitOrTech(item)` EXPORTED

Tells whether an item sits under another item, which makes it a unit or technology rather than a building or group. Marking works differently for the two (Req. 28 to 31).

5.10 `descendants(item)` INTERNAL

Lists every item under an item. Used to pass marks down, and to look at the marks below.

5.11 `sameCount(item)` INTERNAL

Returns an item and the items below it in its line, for as long as they have as many civilisations. A unit's or technology's mark passes down that far (Req. 29, 30).

5.12 `mark(item, kind)` EXPORTED

Toggles a mark. Buildings and groups pass it to all in them; units and technologies only down their line at the same count. Returns the items to show, so the page opens them (Req. 31).

5.13 `boxState(item)` EXPORTED

Returns what an item's box shows. A marked unit or technology shows its own mark, and a dash only if items below carry another mark (Req. 26). Any other item shows `state()`.

5.14 `fit()` EXPORTED

Settles the marks and works out which civilisations fit every one of them. The heart of the civilisation list and of every number. Returns the marked items.

5.15 `markOf(item)` EXPORTED

Returns the mark an item stands for in Filters: the state of everything under it if that is uniform, or else its own mark.

5.16 `lineEnd(item)` EXPORTED

Returns the last item of a line of only children, or nothing if the item branches. Used for marking lines and naming their filters.

5.17 `lineOf(item)` EXPORTED

Returns the whole line an item belongs to, from its top to its last item, or just the item. Filters shows one button per line.

5.18 `chainOf(item)` EXPORTED

Returns an item and each only child below it, down to the first fork or the end. Opening or marking an item opens this chain (Req. 28).

5.19 `filtersOf()` EXPORTED

Works out the Filters buttons: one per marked item whose parent does not already say the same, and one per line and kind of mark. Keeps Filters short.

5.20 `share()` EXPORTED

Finds names that more than one item in the tree uses, so `chipName()` can add the parent's name and keep Filters buttons clear.

5.21 `chipName(item, kind)` EXPORTED

Names a Filters button: "Full" and the name for a fully marked building or group (Req. 24); a line's deepest marked item for units, technologies and buildings with upgrades (Req. 25).

5.22 `unmarkLine(item, kind)`

EXPORTED

Clears the marks behind a Filters button: those of its kind in its line, or the item and all under it. A must lack mark in the line survives clearing must have.

6 js/ui.js

Everything on the screen: drawing the tree and the list, clicks and keys, the language menu, the Options window and the report window.

6.1 `$(id)` EXPORTED

Short for document.getElementById, used wherever the page is touched.

6.2 `clone(id)` EXPORTED

Copies the element inside a template in `index.html`. Repeated markup lives in `index.html` (Rule 2); the code only fills it in.

6.3 `compose(node, template, values)` INTERNAL

Writes a text with placeholders into an element, putting elements such as the source link where their placeholder is. Lets a translated sentence contain a link.

6.4 `plant()` INTERNAL

Draws the tree from the model again, keeping open branches open. Runs at start and when Include Unique and Universal changes.

6.5 `build(list, ul)` INTERNAL

Makes a row for each item from the template, with its arrow, and ids that tie the arrow and the list to the name for screen readers. Calls itself for the items below.

6.6 `isOpen(item)` EXPORTED

Tells whether an item's branch is open, read from its arrow's aria-expanded, which is the one place this is kept.

6.7 `setOpen(item, open)` EXPORTED

Opens or closes an item's branch, and rewrites its count, since a closed item shows two numbers.

6.8 `count(item)` INTERNAL

Writes an item's count of civilisations that fit the filters: its own number, or "own–all" when closed with items under it, and a spoken version for screen readers.

6.9 `expand(item)` INTERNAL

Opens an item and each only child below it, so opening Militia shows its whole line in one step. Used by the arrows and the Right key.

6.10 `shown(item)` EXPORTED

Tells whether an item's row can be seen, that is, whether every branch above it is open.

6.11 `toggle(item, kind)` INTERNAL

Handles a mark from a click, a key, a right-click or a long press: calls `mark()`, opens the way to every row it reached (Req. 28, 31), and redraws.

6.12 `paint()` INTERNAL

Updates every box: its state for the eye, aria-checked for screen readers, and the hidden "must lack" words.

6.13 `update()` INTERNAL

Redraws all that depends on the marks: boxes, the civilisation list and its count, the hint, every item's count and Filters. Runs after every change.

6.14 `find()` INTERNAL

Runs Find: highlights the matching items, announces how many there are, and opens the branches down to them (Req. 20, 21).

6.15 `itemAt(target)` INTERNAL

Returns the tree item a clicked element belongs to, so one listener on the tree serves every row.

6.16 `endPress()` INTERNAL

Cancels a long press in progress and its highlight, when the finger moves or lifts, or the tree is drawn again.

6.17 `wireTree()` INTERNAL

Connects the tree and its tools: keys, clicks, right-clicks, long presses, arrows, Clear, Expand all, Collapse all, Find and the Filters buttons. Runs once at start.

6.18 `options()` EXPORTED

Lists the language menu's buttons. Used by the menu's keys and by the tests.

6.19 `setMenu(open, focus)` EXPORTED

Opens or closes the language menu, hiding the Include setting while it is open so the window keeps its size (Rule 18), and moves focus if asked.

6.20 `buildLanguages()` INTERNAL

Makes one button per language, named and tagged in its own language, so screen readers say each name right (WCAG 3.1.2). Runs once at start.

6.21 `setLanguage(code)` EXPORTED

Shows the page in a language: every text, the page's lang, the menu, the tree's names, the footer's source and date; then redraws. Runs at start and when a language is chosen.

6.22 `label()` INTERNAL

Writes every tree item's name in the current language, and finds names used more than once.

6.23 `showInclude()` INTERNAL

Shows the Include box as ticked or not, for the eye and for screen readers.

6.24 `setInclude(on)` EXPORTED

Switches Include Unique and Universal: saves the choice, draws the tree again keeping marks, and redraws the rest.

6.25 `wireOptions()` INTERNAL

Connects the cogwheel button, the Options window and the Include box, by click and by Space. Runs once at start.

6.26 `choose(code)` INTERNAL

Chooses a language: loads its file if needed, shows it, saves the choice, and puts it in the address (`?lng=`), so a shared link keeps it.

6.27 `wireLanguages()` INTERNAL

Connects the language menu: its button, its options, Up, Down, Home, End and Escape, and closing when focus or a click goes elsewhere. Runs once at start.

6.28 `openReport(title, status)` EXPORTED

Opens the report window, used by the tests and the updater, with a title and a status line, and empties it.

6.29 `report(line)` EXPORTED

Adds a line to the report window.

6.30 `runUpdate()` EXPORTED

Runs the updater: shows its progress, lists what changed, and offers each changed file for download. Opened with `?update`; it changes nothing by itself.

6.31 `start()` EXPORTED

Starts the page: restores the Include choice, draws the tree, builds the language menu, connects every control, shows the chosen language, and reveals the page.

6.32 `showLoadError()` EXPORTED

Shows the message that the data could not be loaded.

7 js/tests/run.js

Runs the tests and shows the report. Loaded only when the address ends in `?test`.

7.1 `merge(runs)` INTERNAL

Combines the results from the main window and the three test frames into one line per test, failed if it failed anywhere and saying where. Sorted by group for the report.

7.2 `framed(size)` INTERNAL

Runs the tests again in a small, phone-sized or tall frame and returns its results. Checks reflow and zoom (WCAG 1.4.10) without a second browser.

7.3 `loadFailed(error)` EXPORTED

Marks the tests as failed when the page could not load, and tells the main window when running in a frame, so CI never waits for a result that cannot come.

7.4 `runTests()` EXPORTED

Runs every test in three rounds, functional, accessibility and maintainability, also in each frame; puts the visitor's settings back; checks the 100-test limit; and shows the report.

8 js/tests/helpers.js

Small tools the test files share: recording results, finding elements, acting like a visitor, and picking sample items.

8.1 `check(name, ok, detail)` EXPORTED

Records one test result. Every test reports this way, and each name is one line in the report.

8.2 `expect(name, offenders)` EXPORTED

Records a test that passes when its list of offenders is empty, naming the first few, so a failure points at the exact items.

8.3 `wait(ms)` EXPORTED

Waits a number of milliseconds. Used to hold a long press.

8.4 `same(a, b)` EXPORTED

Tells whether two lists are equal, item by item. Used to compare lists of civilisations.

8.5 `codes()` EXPORTED

Lists the language codes from `data.json`.

8.6 `keys()` EXPORTED

Lists every item in the full tree.

8.7 `describe(el)` EXPORTED

Names an element briefly, such as button#clear or a tree row's text, for failure messages.

8.8 `rendered(el)` EXPORTED

Tells whether an element can be seen: laid out, not hidden, and not text for screen readers only. The accessibility checks look at what is rendered.

8.9 `everything()` EXPORTED

Lists every element that can be seen on the page.

8.10 `focusables()` EXPORTED

Lists every element that can be seen and reached with the keyboard.

8.11 `textOf(node)` INTERNAL

Reads an element's text as a screen reader would, leaving out parts marked aria-hidden.

8.12 `tidy(text)` EXPORTED

Turns runs of spaces into single spaces and trims the ends.

8.13 `nameFor(el)` EXPORTED

Works out an element's accessible name much as a browser does: from aria-labelledby, aria-label, its label or its text. Lets the tests check that controls are named (WCAG 4.1.2).

8.14 `squash(text)` EXPORTED

Lower-cases a text and keeps only letters and digits, for loose comparisons.

8.15 `visibleTexts(el)` EXPORTED

Lists the visible pieces of text inside an element. Used to find stray placeholders, and to compare names with what can be seen (WCAG 2.5.3).

8.16 `keyDown(target, key)` EXPORTED

Sends a key press to an element, as a visitor's keyboard would.

8.17 `tap(el)` EXPORTED

Clicks an element with mouse pointer events, as a visitor would.

8.18 `type(text)` EXPORTED

Types a text into Find, as a visitor would.

8.19 `pick(code)` EXPORTED

Chooses a language through the Options window and its menu, as a visitor would.

8.20 `reset()` EXPORTED

Returns the page to its start: Find empty, no marks, every branch closed, the menu and the Options window closed, nothing focused. Tests start from here.

8.21 `listed()` EXPORTED

Lists the civilisations the page shows.

8.22 `expected(set)` EXPORTED

Lists the names a set of civilisations should show, translated and sorted, to compare with `listed()`.

8.23 `below(item, top)` EXPORTED

Tells whether an item sits anywhere under another.

8.24 `itemAbove(item)` EXPORTED

Tells whether an item has another item above it, which makes it a unit or technology. The tests work this out on their own, apart from the page's code.

8.25 `fixtures()` EXPORTED

Picks sample items from the current tree, such as a leaf some civilisations have, a building, a line and an optional item. Found from the data, so the tests survive game updates.

9 js/tests/static.js

Tests that read the files themselves, not the page. They run only in the main window.

9.1 `scripts()` EXPORTED

Finds every script by following the imports from `script.js`, noting which load only on demand, so the rule checks cover new scripts by themselves.

9.2 `sourceTests(page, sheet, files)` EXPORTED

Checks the files against the rules: no comments, no CSS or HTML in scripts, where scripts live, at most 400 lines of code, who touches the page, no media, no game data in code.

9.3 `textTests()` EXPORTED

Checks that every language has every page text and group name, and a name for every item and civilisation. Part of the functional round.

9.4 `tagTests()` EXPORTED

Checks that every language has a name and a valid language tag (WCAG 3.1.1, 3.1.2). Part of the accessibility round.

9.5 `dataTests()` EXPORTED

Checks `data.json` for a maintainer's mistakes: items twice in the tree, misplaced flags, missing optional flags, unknown civilisations, plural forms, placeholders, overrides, source details.

9.6 `updaterTests()` EXPORTED

Checks the updater's writer: the files read back without loss, who has what is stored in its shortest form, and names are already tidy, so an update changes only what really changed.

10 js/tests/behaviour.js

Tests of the tree: marking, Filters, counts, opening and closing, the keyboard, and Find.

10.1 `chainOf(item)` INTERNAL

The tests' own version of a branch's chain of only children, kept apart from the page's, so a mistake there cannot hide itself.

10.2 `marking(sample)` INTERNAL

Tests marking by click, X, the menu key and Space; Filters buttons and their names; buildings, groups and lines; and Clear (Req. 3, 17, 22 to 26).

10.3 `filterFocus(sample)` INTERNAL

Tests that clearing a filter moves focus to the next one, and from the last one to the heading, with the hint shown (WCAG 2.4.3). Accessibility round.

10.4 `gestures(sample)` INTERNAL

Tests that a long press with a finger marks must lack, and that a right-click clears it (Req. 3).

10.5 `parents(sample)` INTERNAL

Tests how marks move up the tree: whole lines, optional items, and that no item stays unmarked while everything under it is marked (Req. 27).

10.6 `lines(sample)` INTERNAL

Tests that marking a unit or technology opens its line, and passes the mark down only while the number of civilisations stays the same (Req. 28 to 31).

10.7 `counts(sample)` INTERNAL

Tests the numbers on the items: closed and open, on lines, and with filters (Req. 3).

10.8 `tree()` INTERNAL

Tests Expand all, Collapse all and the arrows, and that opening a branch opens its only children (Req. 18, 19).

10.9 `keyboard(sample)` INTERNAL

Tests moving through the tree with the arrow keys, Home and End, and opening a line with Right (WCAG 2.1.1). Accessibility round.

10.10 `finding(sample)` INTERNAL

Tests that Find highlights, opens to and counts its matches, and says when nothing matches (Req. 20, 21).

10.11 `findKey(sample)` INTERNAL

Tests that Enter in Find jumps to the first match (WCAG 2.1.1). Accessibility round.

10.12 `behaviourTests(sample)` EXPORTED

Runs the tree's functional tests, in order.

10.13 `keyboardTests(sample)` EXPORTED

Runs the tree's accessibility tests: focus after clearing filters, the keyboard, and Enter in Find.

11 js/tests/interface.js

Tests of the language menu, the languages, the Options window and the layout, and the runs of the accessibility audit.

11.1 shownKeys()

INTERNAL

Lists the items currently in the tree, to compare the default tree with the full one.

11.2 uniqueKeys()

INTERNAL

Lists the items inside unique groups, which the default tree leaves out.

11.3 keptPath()

INTERNAL

Finds the path of a leaf to mark while Include changes. Paths survive the tree being built again; item objects do not.

11.4 byPath(path)

INTERNAL

Finds the item with a path in the tree as it is now, after it was built again.

11.5 menuTests(sample)

EXPORTED

Tests the language menu: it is in Options, lists every language, closes cleanly without resizing the window, changes the language, keeps it in the address and keeps marks (Req. 8 to 12).

11.6 languageTests(code, sample)

EXPORTED

Tests one language: the page's text, the tree's names, no placeholders, the footer's source link, and a translated, sorted civilisation list (Req. 4, 5, 11).

11.7 optionTests()

EXPORTED

Tests Include Unique and Universal: off by default, ticking it shows every item and keeps marks, the choice is remembered, and Options opens and closes properly (Req. 7, 13 to 16).

11.8 menuAudit(sample)

INTERNAL

Runs the accessibility audit with the language menu open. This is the audit's first run, so its checks are listed from here.

11.9 menuKeys()

INTERNAL

Tests the language menu's keys: Down opens it at the current language, and Down and End move through it (WCAG 2.1.1).

11.10 languageAudits(code, sample)

INTERNAL

Runs the accessibility audit in one language: as loaded, with filters, and with everything open. The focus checks run in the first language.

11.11 `spacingTests(sample)` INTERNAL

Tests that wider text spacing, as WCAG 1.4.12 describes, makes nothing stick out sideways.

11.12 `optionKeys()` INTERNAL

Tests that Space unticks the Include box, leaving those items out again and keeping marks (WCAG 2.1.1).

11.13 `optionAudits()` INTERNAL

Runs the audit with the Options window open, and with everything included and open.

11.14 `accessibilityTests(sample)` EXPORTED

Runs this file's accessibility tests in the order TESTS.md lists them, then the remaining audits.

12 js/tests/audit.js

The accessibility audit: checks the page as it stands against WCAG 2.2 AAA, wherever a test calls it.

12.1 `rgba(value)` INTERNAL

Reads a CSS colour into its red, green, blue and alpha numbers.

12.2 `blend(top, bottom)` INTERNAL

Blends a see-through colour over the colour below it, to get the colour actually seen.

12.3 `backgroundOf(el)` INTERNAL

Works out the colour behind an element by blending its own and its ancestors' backgrounds. Contrast is measured against this.

12.4 `luminance(rgb)` INTERNAL

Computes a colour's relative luminance, as WCAG defines it.

12.5 `contrast(a, b)` INTERNAL

Computes the contrast ratio of two colours, as WCAG defines it, from 1:1 to 21:1.

12.6 `styles(el)` INTERNAL

Gets an element's styles and those of its ::before, ::after and ::marker parts, so decorations are checked too.

12.7 `focusChecks(list)` INTERNAL

Focuses every control in turn, and checks that it shows a thick outline with 3:1 contrast and that nothing covers it (WCAG 2.4.7, 2.4.12, 2.4.13).

12.8 `audit(label, options)` EXPORTED

Checks the page as it stands: title, languages, landmarks, headings, ids, states, names, contrast, target sizes, no scrolling, reflow, spacing and no media; and focus, if asked.

13 data.json

Everything the page knows, in English: the data source, the languages, the civilisations, the tree of the left pane, which civilisations have each item, items left out on purpose, corrections to names, the texts of the updater and the tests, and the page's text. The updater rewrites it; the tree, the ignore list and the overrides are edited by hand.

```
{"source": {…}, "languages": {…}, "civs": […], "tree": […], "availability": {…},
 "ignore": […], "overrides": {…}, "tools": {…},
 "text": {"ui": {…}, "groups": {…}, "civs": {…}, "items": {…}}}
```

14 `locales/de.json` (example)

One file per other language; this one is German. It holds the language's corrections to names (overrides) and its text: the page's texts, group names, and civilisation and item names. The updater fills in the names; the page's texts are translated by hand. Only the language shown is downloaded.

```
{"overrides": {"Composite Bowman": "Kompositbogenschütze", …},
 "text": {"ui": {"language": "Sprache", "clear": "Zurücksetzen", …},
          "groups": {…}, "civs": {…}, "items": {…}}}
```